Software Security Assignment-1

Name: Harshita Tanwar

AU ID: au828166@uni.au.dk

The application chosen is a Research Workspace Application, which allows users to import dataset links, view datasets and export a formatted version of the datasets to their device. The architecture has three main services:

a) The Application Layer: This is the layer exposed to user input. Users can login based on credentials, import remote links to upload datasets and export formatted datasets using the external package.
b) Resource Layer: This layer contains hardcoded user datasets, handles access to user research databases (as it interacts with the application layer) and stores the exported data.
c) Processing Layer: This layer is responsible for execution of the external package function. An external package is created which converts the user research database into a specific format. The processing service validates the access to database records, formats the data using the external package and saves it to the database.

The GitHub repository for the assignment is linked here: github-repo

A) Why this system & trust boundaries?

The division of the system into these 3 services seemed to be the most appropriate to separate their applications. It is divided based on authentication and authorisation, data storage and processing. Keeping the different processes separate ensure smooth running of the application and it becomes easier to point and mitigate faults, based on what kind of error occurs or what part of the application is affected from certain insecure coding practices or errors.

Application layer is the place where all user input is received (this can be credentials or dataset links) and it is the public entry point. Since it is the only component which faces the clients directly and requires specific checks to handle issues from inputs, it is separate. It handles authorization, authentication (user credentials) and SSRF defence.

The resource layer handles the data. It contains the research datasets (confidential, user-sensitive information) and determines who can read it, based on access determined via authentication at the application layer level.

The processing layer handles the formatting of data with the external package and writing the files to the database. Since this part of the architecture interacts the most with third-party services (writing to the filesystem and accessing the package) it would be apt to make this a separate layer.

Trust Boundary	What crosses it	Untrusted input	Control
Client to API	User credentials, JWT tokens, dataset URLs, JSON outputs (dataset_id, file name)	All user provided input data	Using signed JWT tokens for encryption with expiry, handling URL validation
API to Resource Service	User token (containing username, role and validity) and dataset ID	Dataset ID (needs to be checked for secure access)	The resource service verifies the token and ownership
API to processing service	Dataset ID, output filename, user-specific token	The filename and claim that ownership was checked	Validation with the task token (containing the task, user, dataset and expiry)
API to outside system	Server-side fetch of input URL	The url destination and response	SSRF controls: IP checking and validation, pinning, no redirects
Processing to resource service	The task token, formatted content	The token's scope. Records returned	Check for user access to dataset and whether the task is processing specific or not
Processing to file system	A file path and contents	Output file name (to be written to the filesystem)	Check file name for root path or containment
Processing to external package	Records passed to format summary function	The dependency behaviour	

Where trust is established?

1) **Identity:** The user identity is established solely at the application layer (application API) during the login function based on credentials against the seeded accounts. It then signs a JWT with username and user role along with the expiration (token validity). The algorithm for encryption is SHA256. All the other services also use this token for verification
2) **Authority**: This comes from two types of tokens: a) the user token created during login, it contains the username, role and token validity. The task token is created after checking whether the user has access to specific database records within processing service and contains the task, user name, validity and dataset within user access
3) **Ownership**: This determines who does the dataset records belong to and is exclusively contained in the resource service. It is used in the get_records function in resource service to determine access.
4) **Dependency trust**: This highlights some current weaknesses of the application:
    - The API trusts 8001's 200/403 answer as the ownership verdict.
    - 8002 trusts the API to have issued a legitimate task token, but never verifies it itself. It only forwards it to 8001, which does the verifying.
    - 8001 trusts any token signed with the shared secret.

- Everyone trusts the one shared HS256 key. That is the weak root of trust: any service holding it can mint any identity.

How the three security principles appear in code?

1) **Complete Mediation:** A security principle that ensures that authority is not circumvented in subsequent requests of an object by a subject, by checking for authorization (rights and privileges) upon every request for the object.
   In resource service, the access to database records is checked every time using either the user authentication token (in case of a normal GET request) or using the task token specifically when called by the processing service to generate formatted data. The authentication is checked during saving the exports as well (allowing only processing workers to save the exported datasets).
   The check happens twice for an export (in processing and application service): at the API (the verification call) and again when 8002 fetches the records with the task token. The second check means the data is protected even if the first is skipped.

2) **Principle of Least Privilege**: Design API and roles with minimal permissions by default. While adding vulnerabilities a static all-access key called 'TRUSTED KEY" was created which is a single key for all types of tasks (getting records from resource or exporting datasets). It is replaced by a dataset-specific task token which ensures that processing no longer has standing authority over all data. Since the previous key allowed access to all user records, without any checks
3) **Fail-Safe Defaults**: In the event of missing headers, unparseable IP addresses, or malformed filenames, execution halts closed (default deny). Any missing or malformed token raises HTTP 401; unresolvable/private IP in SSRF validation raises HTTP 403; path traversal characters raise HTTP 400. For SSRF the schemes are an allowlist (http/https only), and one bad resolved IP rejects the whole request. A failed DNS lookup rejects it too. Any suspicious pattern in filename is rejected instead of sanitized and accepted. The required parameters in processing service (task token, dataset ID and output filename) must all be present or the job is refused.

B) How authority and function is carried throughout the workflow. Which checks prevent a permitted service from performing an unauthorized action?

1) The user logs in and obtains a User Session Token (role: user, sub: alice, valid for 2 hours).
2) When Alice requests an export via POST /datasets/{dataset_id}/export, the Application API:
   - Validates Alice's token.

- Queries Resource Service with Alice's token to pre-verify that Alice actually owns {dataset_id}.
- If verified, creates a Scoped Task Token with the task, user, dataset ID and validity, linking the request to a specific task and user who requested access to the records.

3) The application service transmits this created task token to the processing service
4) The processing service uses this task token when requesting records from Resource Service
5) The resource service decodes the token and matches the dataset ID within the token and the processing service task to provide access.

For a dataset READ function:

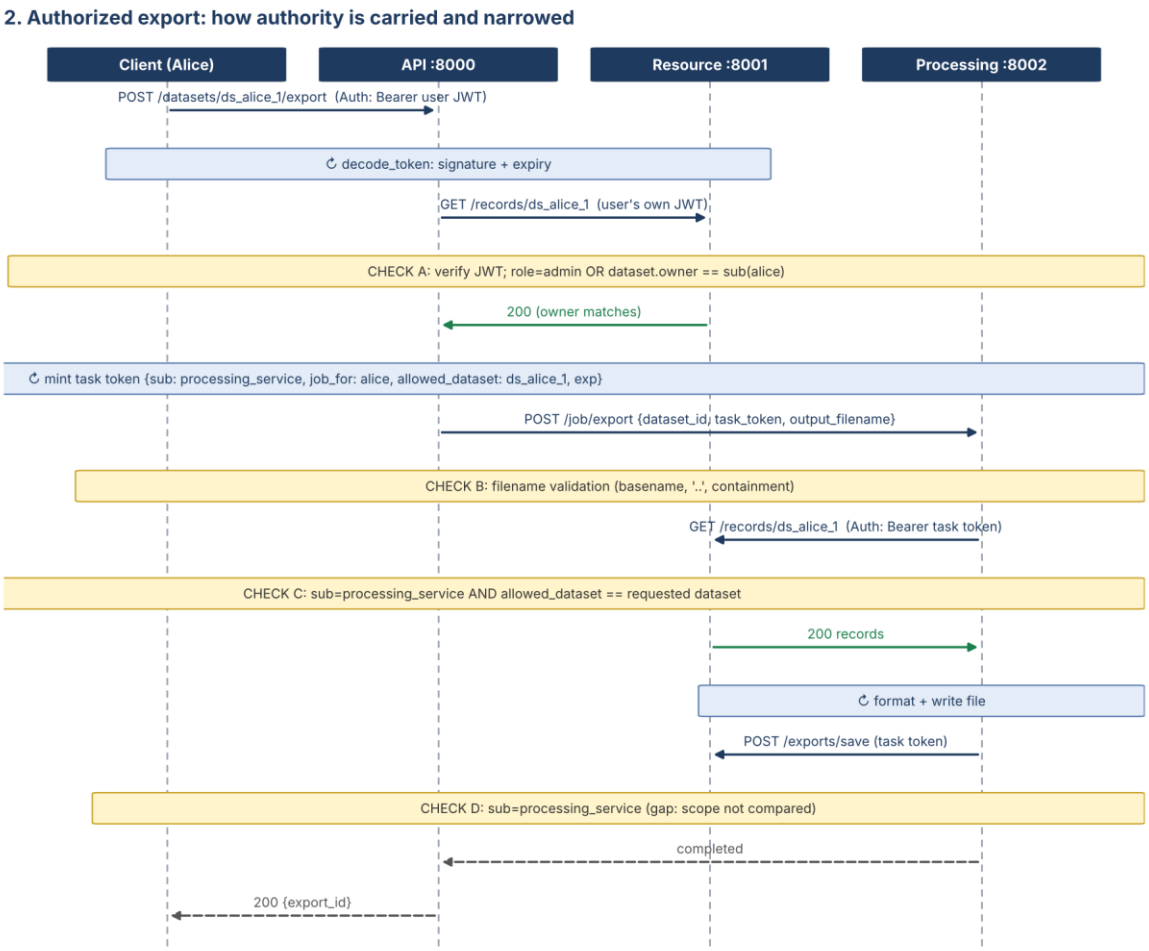
The client sends a GET request (/records/dataset_id) along with the authentication token (containing Bearer and the encrypted token) via the Application API layer to the resource layer to read the dataset. The decode token function at the application layer checks for ownership (whether the dataset record belongs to the user or not) and then passes on the request to the resource layer, which checks for the dataset owner and authentication based on the user role from the token. Admin can read all the user datasets; a user is only restricted to their own datasets.

Audit Log access:

This is a privileged function allowing only the admin to have access to audit logs. The client sends a GET request using /admin/audit-logs with the Auth Bearer token, and at the application layer, the decode token function checks for access based on user role, an error 403 is raised if the user does not carry the admin role. The checks enforced includes Role-Based Access Control (using the token to check role) and ordinary users like (alice, bob) attempting to call this route receive HTTP 403 Forbidden, enforcing Least Privilege.

Import Dataset with link:

This function again involves the user sending a POST request to import a dataset link at the application layer, which carries the authentication token. At application layer the authentication token is decoded and the input URL is validated. The checks enforced include: Accepting only http or http, resolves both IPv4 and IPv6 addresses and pins outgoing TCP handshake to the exact validated numeric IP address.



C) Which protections came from your language/runtime, libraries, application logic, and deployment configuration?

Running in Python inherently eliminates low-level memory corruption vulnerabilities such as buffer overflows, use-after-free, or arbitrary memory pointers. Running three independent Python processes on distinct local network ports enforces operating system process boundaries; a runtime crash, memory leak, or unhandled exception in processing service cannot crash the other services.

Used os.path.abspath() to resolve symbolic links, relative references, and canonical file paths on the underlying host operating system. Utilized Python's standard socket.getaddrinfo() to perform dual-stack DNS resolution directly through the OS resolver. Used the standard ipaddress library to perform mathematical subnet range checks (is_private, is_loopback, is_reserved, is_link_local) across both IPv4 and IPv6 families without relying on brittle regex string matching.

Third-party libraries involved using pyjwt which enforces digital signature verification using HMAC-SHA256 (HS256), guaranteeing token integrity and preventing payload tampering. Provides strict input structure and HTTP header extraction (authorization: str = Header(...)), ensuring requests missing required credentials fail before route handlers are invoked. It also

centralizes exception handling via standard HTTPException primitives to prevent leaking raw Python stack traces, file paths, or internal line numbers to external clients.

D) Document the planted/known vulnerabilities

1) CWE-918 Server-Side Request Forgery

SSRF occurs when an attacker can influence a server into making a network request to a destination that the attacker should not be able to access directly. The attacker does not necessarily access the target directly; the vulnerable server is induced to make the request on the attacker's behalf. (from course material)

Real-world scientific research workspaces often allow the users to directly import dataset links from the web. Since the application layer in the system is exposed to all kinds of external inputs, this specific vulnerability was planted to see how can external web links be handled.

This violates the system, as instead of fetching external research data, an attacker abuses the edge server's internal network to reach private backend services (such as the Resource Service at (http://127.0.0.1:8001)) that were never intended to be reachable directly from the outside.

Hence, CWE-918 maps to Information Disclosure and Elevation of Privilege in this case. The attacker tricks the server into querying internal services, cloud metadata endpoints or status endpoints. The server reads internal data and reflects it back to the client in the response payload (as seen in resp.text[:100]), exposing internal network topology, tokens, or private resources.

By routing requests through the application layer, the attacker can assume the privileges of the internal network administrators bypassing trust boundaries. It could also lead to a DoS attack as the attacker can force the server to connect to unresponsive internal IP addresses, causing thread exhaustion and connection timeouts, or flood internal microservices with unauthorized traffic.

This is an implementation-level problem since the endpoint took untrusted user input and passed it directly to an outbound network API without validating the destination.

The code failed to verify the protocol scheme, failed to resolve the hostname to an IP address, and failed to check whether the resulting IP belonged to a loopback, private, or reserved subnet before dispatching the request.

Mitigations made:

- The URL scheme is restricted to http or https. Any unexpected schemes like file, fttp, gopher are rejected before network resolution
- Instead of checking only IPv4 via gethostbyname(), the code queries all address families (IPv4 and IPv6) for every IP associated with the target domain

- Every returned IP is evaluated using Python's ipaddress module against restricted ranges like loopback, link local and reserved.
- The HTTP request is made directly to the validated numeric IP rather than the hostname string, while preserving the original hostname in the HTTP Host header. This prevents the HTTP client from issuing a second DNS query that an attacker could manipulate via DNS rebinding.

2) **Application-Specific Vulnerability**

In distributed architectures, front-end gateways dispatch heavy jobs to background worker services. This vulnerability uses full 3-tier pipeline (Application API → Processing Service → Resource Service). The vulnerability cannot exist in isolation inside a single service; it arises purely from how authority is across service boundaries.

Alice cannot access Bob's data directly at the Resource Service because of access controls (or tenant boundaries). However, she can ask the Application API to tell the Processing Service to process Bob's dataset. The Processing Service, with its internal credentials, obediently Bob's data and provides the exported file to Alice. The Processing Service is used for its elevated authority on behalf of an unauthorized actor. This vulnerability can come under 3 STRIDE categories: Elevation of privilege, Repudiation and Information Disclosure.

The user (alice) has access to only her data. Using the processing service to access and export another user's data is an example of elevating user privilege to admin. Since all the content can be accessed by another user, this causes information disclosure. And since the processing service would be logged as requesting access to other user data. So, this completely obscure the identity of the actual caller or unauthorized actor.

Mitigation:

- Before any background worker or queue is notified, the Application layer validates that the requesting caller actually possesses the right to access dataset. It queries the Resource Service directly using the client's own bearer identity token. If Alice attempts to export Bob's data, the call fails immediately at the edge with HTTP 403 Forbidden, aborting the pipeline before delegating work.
- Instead of handing the background worker a permanent, static global key (TRUSTED KEY), the Application API creates a short-lived cryptographic Task Token (HS256 JWT).
- When the Processing Service makes its internal call to GET /records/{dataset_id} on the Resource Service, it presents the task token. The Resource Service decodes the token, checks the signature and expiration, and verifies that token["allowed_dataset"] == dataset_id.

3) **CWE-22: Path Traversal**

Path Traversal occurs when attacker-controlled input is used to construct a file or directory path and allows the attacker to access resources outside the intended directory or resource boundary. (from the provided course material)

In case of an application, where the user is writing a file to the system and can name it as they like, and if that filename is accepted without sanitization, the filesystem write operation becomes an attack vector against the host OS. Hence a CWE-22 vulnerability would be appropriate to expand upon.

The primary threat posed here is tampering. By injecting relative path sequences (../), an attacker escapes workspace-exports folder and writes to arbitrary directories on the host. They can overwrite critical configuration files, source code, .env files, tampering with the integrity of the service or host OS. Overwriting certain scripts can also allow an unprivileged user to execute arbitrary commands at the service process's privilege level, causing Elevation of privilege as well.

Mitigation:

- os.path.basename(): Strips directory prefixes and separators. If output filename is ../../test.txt, os.path.basename yields test.txt. If the cleaned string does not match the original string, an evasion attempt is flagged.
- Canonicalization via os.path.abspath(): Resolves symbolic links and relative path sequences into a fully qualified canonical path on the filesystem.
- Confirm that the target path physically resides inside the designated boundary folder, preventing escapes even if path parsing behaves unexpectedly.

4) **CWE-862: Missing Authorization**

Missing Authorization occurs when software performs a protected operation or provides access to a resource without adequately verifying that the requesting user or process is authorized to perform that action.

It addresses tenant isolation between users (alice and bob), demonstrating the core security requirement that one user cannot view another's sensitive assets.

The primary threat here is information disclosure, an unprivileged tenant reads confidential data belonging to another tenant simply by enumerating or supplying an identifier (dataset record in this case). It can also map to elevation of privilege since the allowing a user to access other user data, elevates their privileges to that of admin.

Mitigation:

- Every single data retrieval (GET /records/{dataset_id}) and export registration (POST /exports/save) requires an explicit Authorization header containing an authentic token. A user token and task token are two levels of check which are done

when calling the processing service. No internal bypass is permitted, even if requests originate from local network interfaces.

- Missing headers, invalid signatures, expired tokens, or unassigned permissions fail closed immediately (HTTP 401 or HTTP 403), terminating execution before any dataset records are returned.